

Problem: Construct Binary Tree from Preorder and Inorder

Goal:

Given two lists:

- **Preorder** (Root → Left → Right)
 - **Inorder** (Left → Root → Right)
- Construct the binary tree.
-

Procedure (Step-by-Step)

1. **Start with Preorder[0]** — this is the **root** of the tree.
 2. Find this root in the **inorder** list.
 - Elements **left of root in inorder** → left subtree
 - Elements **right of root in inorder** → right subtree
 3. Use the lengths of left and right subtrees to split the preorder list.
 4. **Recursively** build left and right subtrees using the respective portions of preorder and inorder.
-

Example Format:

For each example:

- Show the input
 - Explain the split
 - Show the construction step by step
 - Display the final tree
-

Example 1

► Input:

- **Preorder:** A B D E C F
 - **Inorder:** D B E A F C
-

□ Steps:

1. **Root = A** (pre[0])
2. Find A in inorder $\rightarrow$ index 3
 - Left inorder: D B E
 - Right inorder: F C

Split preorder accordingly:

- Left pre: B D E
 - Right pre: C F
-

$\rightarrow$ Left Subtree (B D E)

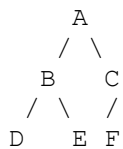
- Root = B
- Inorder: D B E $\rightarrow$ B at index 1
 - Left = D
 - Right = E

$\rightarrow$ Right Subtree (C F)

- Root = C
 - Inorder: F C $\rightarrow$ C at index 1
 - Left = F
 - Right = -
-

✓ Final Tree:

mathematica
CopyEdit



♣ Example 2

► Input:

- Preorder: 1 2 4 5 3 6

- Inorder: 4 2 5 1 3 6

□ Steps:

1. Root = 1
2. 1 in inorder $\rightarrow$ index 3
 - Left inorder: 4 2 5
 - Right inorder: 3 6

Split pre:

- Left pre: 2 4 5
- Right pre: 3 6

Left Subtree (2 4 5):

- Root = 2 $\rightarrow$ Inorder: 4 2 5
 - 2 is at index 1 $\rightarrow$ left = 4, right = 5

Right Subtree (3 6):

- Root = 3 $\rightarrow$ Inorder: 3 6 $\rightarrow$ 3 is index 0 $\rightarrow$ right = 6

✓ Final Tree:

markdown
CopyEdit

```

      1
     / \
    2   3
   / \   \
  4  5   6
  
```

♣ Example 3

► Input:

- Preorder: M N O P Q
- Inorder: O N P M Q

□ Steps:

1. Root = M
2. M in inorder $\rightarrow$ index 3
 - Left = O N P
 - Right = Q

Left preorder: N O P

Right preorder: Q

Left Subtree:

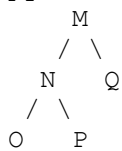
- Root = N
- Inorder: O N P $\rightarrow$ N at index 1 $\rightarrow$ left = O, right = P

Right = Q (leaf)

✓ Final Tree:

css

CopyEdit



Problem: Construct Binary Tree from Postorder and Inorder

🎯 Given:

- **Postorder** (Left → Right → Root)
 - **Inorder** (Left → Root → Right)
- Construct the binary tree.
-

□ Step-by-Step Construction Procedure

1. The **last element** in the postorder list is the **root** of the tree.
 2. Find the **index of this root** in the inorder list.
 - Elements **to the left** in inorder → left subtree
 - Elements **to the right** in inorder → right subtree
 3. Use the lengths of these left/right parts to divide the postorder list.
 4. **Recurse** on left and right subtrees.
 5. Build tree **from bottom up**.
-

✓ Key Rule:

We build the tree **backward from postorder** (last item is root) and divide based on **inorder** structure.

Now let's do **10 detailed examples**. 🔍

♣ Example 1

➤ Input:

- **Inorder:** D B E A F C
 - **Postorder:** D E B F C A
-

□ Steps:

1. $\text{Post}[-1] = A \rightarrow \text{root}$
2. A in inorder at index 3
 - Left inorder: D B E
 - Right inorder: F C

Split postorder accordingly:

- Left post: D E B
- Right post: F C

→ Left Subtree:

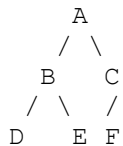
- Root = B (last in left postorder)
- Inorder: D B E $\rightarrow$ B at index 1 $\rightarrow$ left = D, right = E

→ Right Subtree:

- Root = C
- Inorder: F C $\rightarrow$ C at index 1 $\rightarrow$ left = F

✓ Tree:

mathematica
CopyEdit



♣ Example 2

➤ Input:

- Inorder: 4 2 5 1 6 3
- Postorder: 4 5 2 6 3 1

□ Steps:

1. Root = 1 (last in postorder)
2. 1 in inorder at index 3 $\rightarrow$ left: 4 2 5, right: 6 3

Split post:

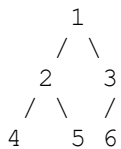
- Left: 4 5 2 $\rightarrow$ root = 2
- Right: 6 3 $\rightarrow$ root = 3

2's children: 4 and 5

3's child: 6

✓ **Tree:**

markdown
CopyEdit



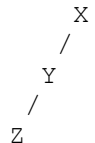
♣ Example 3

- Inorder: Z Y X
- Postorder: Z Y X

1. Root = X
2. Inorder: left = Z Y
 $\rightarrow$ Root = Y
 $\rightarrow$ Left = Z

✓ **Tree:**

markdown
CopyEdit



♣ Example 4

- Inorder: B A C
- Postorder: B C A

1. Root = A
2. Left = B, Right = C

✓ Tree:

```
css
CopyEdit
  A
 / \
B   C
```

♣ Example 5

- Inorder: A B C D
- Postorder: A C D B

1. Root = B
2. Inorder: A (left), C D (right)
→ Right root = D → left = C

✓ Tree:

```
css
CopyEdit
  B
 / \
A   D
    /
   C
```

♣ Example 6

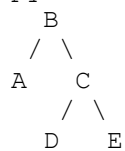
- Inorder: A B D E C
- Postorder: A E D C B

Root = B
 Left = A
 Right = D E C → C is root, D and E are left and right

✓ Tree:

mathematica

CopyEdit



♣ Example 7

- Inorder: 3 2 4 1 5
- Postorder: 3 4 2 5 1

Root = 1

Left = 3 2 4 $\rightarrow$ Root = 2

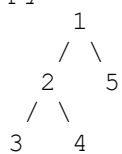
$\rightarrow$ Left = 3, Right = 4

Right = 5

✓ Tree:

markdown

CopyEdit



♣ Example 8

- Inorder: 8 4 9 2 10 5 1
- Postorder: 8 9 4 10 5 2 1

Root = 1

Left = 8 4 9 2 10 5 $\rightarrow$ Root = 2

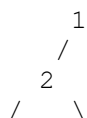
$\rightarrow$ Left = 8 4 9 $\rightarrow$ Root = 4 $\rightarrow$ 8 & 9

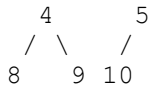
$\rightarrow$ Right = 10 5 $\rightarrow$ Root = 5, Left = 10

✓ Tree:

markdown

CopyEdit





♣ Example 9

- Inorder: 7 3 8 1 9 4
- Postorder: 7 8 3 9 4 1

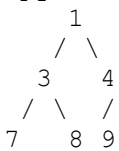
Root = 1

Left = 7 3 8 → Root = 3 → 7 and 8

Right = 9 4 → Root = 4 → Left = 9

✓ Tree:

markdown
CopyEdit



♣ Example 10

- Inorder: 2 1 3 4
- Postorder: 2 4 3 1

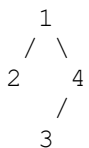
Root = 1

Left = 2

Right = 3 4 → 4 is root, 3 is left

✓ Tree:

markdown
CopyEdit



When is it possible to build a unique binary tree from Preorder and Postorder?

If the tree is a **Full Binary Tree**:

- Every node has either **0 or 2 children**
 - Then a **unique** tree can be constructed from Preorder and Postorder traversals
-

□ Construction Logic (Procedure)

Given:

- **Preorder**: Root $\rightarrow$ Left $\rightarrow$ Right
- **Postorder**: Left $\rightarrow$ Right $\rightarrow$ Root

Steps:

1. Start with the first element of **preorder**, that's the **root**.
 2. The **next element** in preorder is the **left child**.
 3. Search this child in the **postorder** array to determine the boundary of the left subtree.
 4. Recur for left and right subtrees using the split positions in postorder.
 5. Use indexes or pointers to keep track of the current root in preorder and postorder bounds.
-

✧✧ Let's Now Walk Through 5 Examples

♣ Example 1

Preorder: A B D E C F G

Postorder: D E B F G C A

Step-by-step:

1. $\text{Pre}[0] = A \rightarrow \text{root}$
2. $\text{Pre}[1] = B \rightarrow \text{left child of A}$
 - B is at index 2 in postorder $\rightarrow$ subtree size = 3
3. Recur for:

- Left subtree: Pre[1:4], Post[0:2] → B D E
- Right subtree: Pre[4:], Post[3:5] → C F G

Build left:

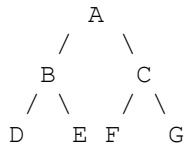
- B (root), D (left), E (right)

Build right:

- C (root), F (left), G (right)

✓ Final Tree:

mathematica
CopyEdit



🌲 Example 2

Preorder: 1 2 4 5 3 6 7

Postorder: 4 5 2 6 7 3 1

Steps:

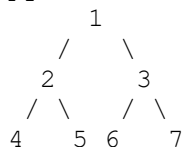
1. Root = 1
2. Next = 2 → at index 2 in postorder → subtree size = 3
3. Left: Pre[1:4], Post[0:2] → 2 4 5
4. Right: Pre[4:], Post[3:5] → 3 6 7

Left: 2 → 4 (left), 5 (right)

Right: 3 → 6 (left), 7 (right)

✓ Tree:

markdown
CopyEdit



♣ Example 3

Preorder: x y z

Postorder: z y x

Steps:

1. Root = X
2. Next = Y $\rightarrow$ at index 1 in postorder
 - Left subtree size = 2
3. Left: Pre[1:], Post[0:1] $\rightarrow$ Y Z

Y is root, Z is child

No right subtree

✓ Tree:

markdown
CopyEdit

```
      X
     /
    Y
   /
  Z
```

♣ Example 4

Preorder: A B C D

Postorder: C D B A

Steps:

1. Root = A
2. Next = B $\rightarrow$ at index 2 in postorder
3. Pre[1:4] = B C D $\rightarrow$ post[0:2] = C D B

Root = B

- Next = C $\rightarrow$ C is at index 0 $\rightarrow$ left = C
- Next = D $\rightarrow$ right = D

✓ Tree:

CSS

CopyEdit



♣ Example 5

Preorder: P Q R S T

Postorder: R Q T S P

Steps:

1. Root = P
2. Next = Q $\rightarrow$ index 1 in postorder
 $\rightarrow$ Left subtree size = 2

Left: Q R

Right: S T

Q $\rightarrow$ left = R
S $\rightarrow$ right = T

✓ Tree:

css

CopyEdit

